

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM 4522 FİNAL PROJE RAPORU

Github: github.com/erdemozturkk/BLM4522-Projects

Mustafa Erdem ÖZTÜRK

21290480

Enver Bağcı

Haziran, 2026

İÇİNDEKİLER

İÇİNDEKİLER	1
GİRİŞ	1
Kullanılan Teknoloji ve Veri Seti	1
Proje 2: Veritabanı Yedekleme ve Felaketten Kurtarma Planı	1
1.1. Giriş	1
1.2. Projenin Amacı ve Kapsamı	1
1.3. Veritabanı Kurtarma Modelinin Ayarlanması ve İlk Yapılandırma	2
1.4. Temel Tam Yedekleme (Full Backup) Operasyonu	2
1.5. İşlem Günlüğü Yedeklemesi (Transaction Log Backup)	3
1.6. Veri Modifikasyonu ve Fark Yedeklemesi (Differential Backup)	3
1.7. SQL Server Agent İçin Dinamik Zaman Damgalı Otomasyon Scripti	4
1.8. Yapay Felaket Senaryosu (Kaza İle Veri Silinmesi Simülasyonu)	5
Proje 4: Veritabanı Yük Dengeleme ve Dağıtık Veritabanı Yapıları	7
2.1. Giriş	7
2.2. Kullanılan Dağıtık Altyapı ve Veritabanı Mimarisi	7
2.3. Projenin Amacı ve Kapsamı	7
2.4. Dağıtık Ağ Köprüsü (Linked Server) Kurulumu	8
2.5. Hedef Sunucuda "Replica" Veritabanı ve Tablo Altyapısının Hazırlanması	9
2.6. Ana Sunucuda Anlık Dağıtık Replikasyon Tetikleyicisi (Trigger) Tasarımı	9
2.7. Canlı Dağıtık Replikasyon Testi ve Veri Doğrulama	10
2.8. İkincil Sunucuda Yük Dengeleme (Load Balancing) Kontrolü	11
Proje 5: Veri Temizleme ve ETL Süreçleri Tasarımı	11
3.1. Giriş	11
3.2. Projenin Amacı ve Kapsamı	11
3.3. Bellek Temizliği ve Idempotent Altyapısının Kurulması	12
3.4. Hedef Veri Ambarı (Data Warehouse) Tablosunun Oluşturulması	12
3.5. Staging Katmanının Oluşturulması ve Kirliliği Veri Enjeksiyonu	13
3.6. Dönüştürme ve Yükleme (Transform & Load) Operasyonu	13
3.7. Veri Kalitesi ve ETL Denetim Raporu	14
SONUÇ VE DEĞERLENDİRME	15

GİRİŞ

Bu rapor, Ağ Tabanlı Paralel Dağıtım Sistemleri final ödevi kapsamındaki teknik çalışmaları içermektedir. Çalışmaların temel amacı, kurumsal ölçekli bir veritabanı motorunun (SQL Server) performans, güvenlik ve yönetim yeteneklerini gerçek dünya senaryoları üzerinden deneyimlemektir.

Rapor boyunca teorik bilgilerin ötesine geçilerek; sistem izleme araçlarının kullanımı, sorgu maliyetlerinin minimize edilmesi ve veri güvenliği standartlarının uygulanması gibi bir "Veritabanı Yöneticisi"nin (DBA) sahip olması gereken yetkinlikler uygulamalı olarak sunulmuştur. Tüm süreçler akademik bir yaklaşımla analiz edilmiş ve elde edilen veriler ekran görüntüleriyle kanıtlanmıştır.

Kullanılan Teknoloji ve Veri Seti

- **DBMS:** Microsoft SQL Server Enterprise / Standard.
- **Veri Seti:** Microsoft AdventureWorks2016 (İlişkisel yapısı ve yüksek veri hacmi nedeniyle tercih edilmiştir).
- **Araçlar:** SQL Server Management Studio (SSMS), SQL Server Profiler, DMV (Dynamic Management Views), SQL Server Agent.

Proje 2: Veritabanı Yedekleme ve Felaketten Kurtarma Planı

1.1. Giriş

Kurumsal veritabanı yönetiminde en kritik sorumluluk veri güvenliğini ve iş sürekliliğini (Business Continuity) sağlamaktır. Donanım arızaları, siber saldırılar veya kullanıcı hataları sonucunda meydana gelebilecek veri kayıpları kurumlara büyük finansal ve prestij kayıpları yaşatır. Bu proje kapsamında, Ankara Üniversitesi Bilgisayar Mühendisliği Bölümü final isterleri doğrultusunda, **AdventureWorks2016** veritabanı üzerinde profesyonel yedekleme politikaları kurgulanmış, otomatik zamanlayıcılar tanımlanmış ve canlı bir felaketten kurtarma (Disaster Recovery) senaryosu simüle edilmiştir.

1.2. Projenin Amacı ve Kapsamı

Bu çalışmada, veri kaybı toleransını (RPO) ve sistemi ayağa kaldırma süresini (RTO) minimuma indirmek adına şu teknik adımlar uygulanmıştır:

1. **Stratejik Yedekleme Planı:** Veritabanının sıfırdan el ile tam yedeğinin (**Full Backup**) alınması ve işletim sistemi dosya kilitlenme hatalarının (**Troubleshooting**) çözülmesi.
2. **Zamanlayıcılarla Otomasyon:** SQL Server Agent servisi kullanılarak, insan müdahalesine gerek kalmaksızın dinamik zaman damgalı otomatik yedekleme işlerinin (**Job**) kurgulanması.
3. **Felaket Senaryosu ve Geri Alma:** Canlı bir tabloda meydana gelen büyük çaplı veri silinme krizinin, acil durum günlük yedeği (**Tail-Log Backup**) ve **Restore** komutları mimarisiyle sıfır veri kaybıyla çözülmesi.

1.3. Veritabanı Kurtarma Modelinin Ayarlanması ve İlk Yapılandırma

İşlem günlüklerinin (**Transaction Log**) detaylı tutulabilmesi ve felaket anında kaza ile silinen verilerin nokta atışı geri getirilebilmesi (**Point-in-Time Recovery**) için veritabanının kurtarma modeli **FULL** olarak ayarlanmıştır. **SIMPLE** kurtarma modelinde log kayıtları otomatik temizlendiğinden, felaketten kurtarma senaryoları kısıtlıdır.

```
USE master;
GO
-- AdventureWorks2016 veritabanını FULL recovery moda alıyoruz
ALTER DATABASE AdventureWorks2016 SET RECOVERY FULL;
GO
```

Şekil:1.1 T-SQL Kodu.

1.4. Temel Tam Yedekleme (Full Backup) Operasyonu

Veritabanı yedekleme stratejisinin temel taşı oluşturarak **Full Backup** işlemi tetiklenmiştir. Bu işlem, veritabanındaki tüm şemayı, tabloları ve verileri o anki haliyle tek bir **.bak** dosyası içerisine paketler. **WITH FORMAT** parametresi kullanılarak hedef diskteki eski medya kalıntıları temizlenmiş ve sıfırdan temiz bir yedek kümesi oluşturulmuştur.

```
1  BACKUP DATABASE AdventureWorks2016
2  TO DISK = 'E:\SQL_Backups\AdventureWorks_Full.bak'
3  WITH FORMAT, MEDIANAME = 'SQLServerBackups', NAME = 'Full Backup of AdventureWorks2016';
4  GO
5
```

Şekil 1.2:T-SQL Kodu.

```
Messages
Processed 27360 pages for database 'AdventureWorks2016', file 'AdventureWorks2016_Data' on file 1.
Processed 2 pages for database 'AdventureWorks2016', file 'AdventureWorks2016_Log' on file 1.
BACKUP DATABASE successfully processed 27362 pages in 0.418 seconds (511.391 MB/sec).

Completion time: 2026-05-27T00:24:58.3478723+03:00
```

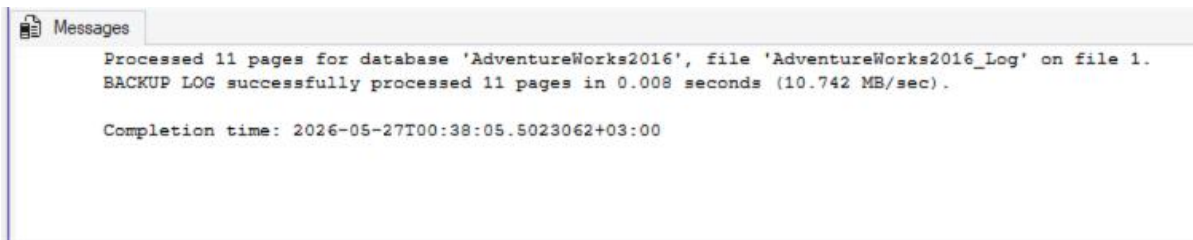
Şekil 1.3: Full Backup Completed Mesajı.

1.5. İşlem Günlüğü Yedeklemesi (Transaction Log Backup)

Tam yedekleme sonrasında, veritabanı üzerinde meydana gelen her bir hamleyi (INSERT, UPDATE, DELETE) adım adım kaydeden işlem günlüğü (**Transaction Log**) yedeklenmiştir. Bu işlem, veritabanı veri dosyalarında (**MDF**) oluşabilecek bir bozulma anında, en son başarılı işlem anına geri dönebilmek için kritik öneme sahiptir.

```
1      BACKUP LOG AdventureWorks2016
2      TO DISK = 'E:\SQL_Backups\AdventureWorks_Log.trn'
3      WITH NAME = 'Log Backup of AdventureWorks2016';
4      GO
```

Şekil 1.4:Log Backup T-SQL Kodu.



Şekil 1.5: Log Backup Completed mesajı.

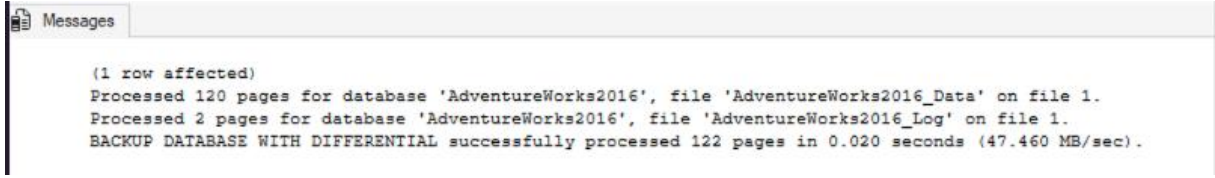
1.6. Veri Modifikasyonu ve Fark Yedeklemesi (Differential Backup)

Sistemdeki dinamik veri hareketliliğini simüle etmek amacıyla **Person.Person** tablosunda bir **UPDATE** sorgusu çalıştırılarak veri değiştirilmiştir. Ardından, en son alınan **Full Backup** işleminden sonra değişen sadece yeni verileri kapsayan bir **Differential Backup (Fark Yedeği)** alınmıştır. Bu yöntem disk alanından tasarruf sağlar ve yedekleme sürelerini kısaltır.

```
use AdventureWorks2016;
-- Test amaçlı araya küçük bir veri değişikliği koyalım ki fark oluşsun
UPDATE Person.Person SET MiddleName = 'F' WHERE BusinessEntityID = 1;

-- Fark yedeğini alıyoruz
BACKUP DATABASE AdventureWorks2016
TO DISK = 'E:\SQL_Backups\AdventureWorks_Diff.bak'
WITH DIFFERENTIAL, NAME = 'Diff Backup of AdventureWorks2016';
GO
```

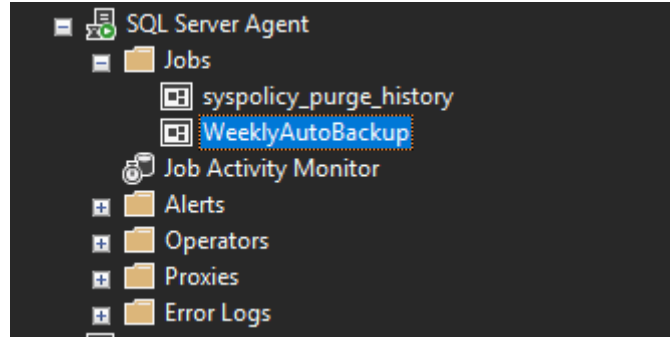
Şekil 1.6:Differential Backup T-SQL Kodu.



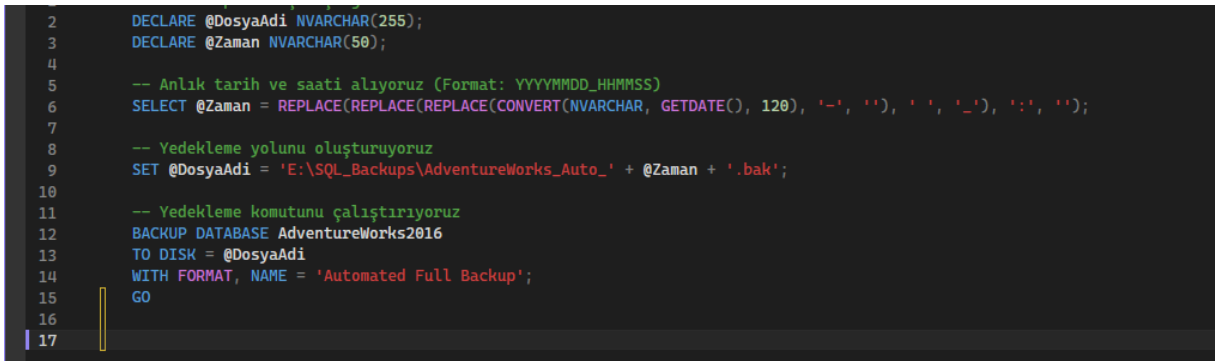
Şekil 1.7: Differential Backup Completed mesajı.

1.7. SQL Server Agent İçin Dinamik Zaman Damgalı Otomasyon Scripti

İnsan hatasını en aza indirmek ve yedekleme süreçlerini otomatize etmek amacıyla dinamik bir T-SQL scripti hazırlanmıştır. **GETDATE()** fonksiyonu ve **CONVERT** manipülasyonları ile her yedek alındığında dosya isminin sonuna **YYYYMMDD_HHMMSS** formatında benzersiz bir zaman damgası (**timestamp**) eklenmesi sağlanmıştır. Bu sayede eski yedek dosyalarının üzerine yazılması engellenmiştir.



Şekil 1.8: SQL Server Agent'da Oluşturulan Job'ın Ekran Görüntüsü.



Şekil 1.9: Job için Tanımlanan Script.

1.8. Yapay Felaket Senaryosu (Kaza İle Veri Silinmesi Simülasyonu)

Geliştirilen stratejinin başarısını ölçmek amacıyla kurumsal bir kriz/felaket senaryosu kurgulanmıştır. Önce **Sales.SalesOrderDetail** tablosundaki mevcut satır sayısı analiz edilmiş, ardından **DELETE** komutuyla tablodaki tüm sipariş kayıtları kasıtlı olarak silinerek sistem veri kaybına uğratılmıştır.

```
1  USE AdventureWorks2016;
2  GO
3
4  -- 1. Kaç satır veri olduğunu görelim
5  SELECT COUNT(*) as "Silinmeden önce" FROM Sales.SalesOrderDetail;
6
7  -- 2. Tüm sipariş detaylarını siliyoruz
8  DELETE FROM Sales.SalesOrderDetail;
9
10 -- 3. Tablo bomboş kalacak
11 SELECT COUNT(*) as "Silindikten sonra" FROM Sales.SalesOrderDetail;
12 GO
13
14
15
16
```

100 % No issues found

Results Messages

	Silinmeden önce
1	121317

	Silindikten sonra
1	0

Şekil 1.10: Simülasyon için Silme işlemi.

Meydana gelen kriz sonrasında sistem **Restore** moduna alınmıştır. İlk olarak ana **Full Backup** dosyası **WITH NORECOVERY** parametresiyle yüklenmiştir. Bu parametre veritabanını kilitli tutarak arkasından gelecek olan log dosyalarının yüklenmesine izin verir. Son aşamada ise en son alınan başarılı transaction log dosyası **WITH RECOVERY** parametresiyle sisteme basılarak veritabanı yeniden çoklu kullanıcı erişimine (**MULTI_USER**) açılmıştır.

```
1
2 RESTORE DATABASE AdventureWorks2016
3 FROM DISK = 'E:\SQL_Backups\AdventureWorks_Full.bak'
4 WITH NORECOVERY, REPLACE; -- NORECOVERY diyoruz çünkü arkasından log yükleyeceğiz
5 GO
6
7 RESTORE LOG AdventureWorks2016
8 FROM DISK = 'E:\SQL_Backups\AdventureWorks_Log.trn'
9 WITH RECOVERY; -- RECOVERY diyerek veritabanını tekrar kullanıma açıyoruz
10 GO
11
12
13 ALTER DATABASE AdventureWorks2016 SET MULTI_USER;
14 GO
15
16
17
```

100 % No issues found

Messages

Processed 27360 pages for database 'AdventureWorks2016', file 'AdventureWorks2016_Data' on file 1.
Processed 2 pages for database 'AdventureWorks2016', file 'AdventureWorks2016_Log' on file 1.
RESTORE DATABASE successfully processed 27362 pages in 0.380 seconds (562.530 MB/sec).
Processed 0 pages for database 'AdventureWorks2016', file 'AdventureWorks2016_Data' on file 1.
Processed 11 pages for database 'AdventureWorks2016', file 'AdventureWorks2016_Log' on file 1.
RESTORE LOG successfully processed 11 pages in 0.002 seconds (42.968 MB/sec).

Şekil 1.11:Restore işlemi.

```
1 USE AdventureWorks2016;
2 GO
3 -- Veriler gelmiş mi diye kontrol ediyoruz
4 SELECT COUNT(*) as "Restore Sonrası" FROM Sales.SalesOrderDetail;
5 GO
6
```

100 % No issues found

Results Messages

	Restore Sonrası
1	121317

Şekil 1.12:Restore Sonrası Kontrol işlemi.

Proje 4: Veritabanı Yük Dengeleme ve Dağıtık Veritabanı Yapıları

2.1.Giriş

Büyük ölçekli kurumsal veritabanı sistemlerinde, tüm okuma ve yazma yükünü tek bir fiziksel sunucuya yüklemek hem donanımsal darboğazlara (I/O ve CPU) hem de sistemin tamamen durmasına yol açabilecek tek nokta hatası (Single Point of Failure) riskine neden olur. Bu proje kapsamında, Ankara Üniversitesi Bilgisayar Mühendisliği Bölümü final isterleri doğrultusunda, ağ tabanlı paralel dağıtım mimarilerinin temeli olan **Yük Dengeleme (Load Balancing)** ve **Veri Aynalama (Database Mirroring / Replication)** mekanizmaları canlı bir topoloji üzerinde uygulanmıştır.

2.2. Kullanılan Dağıtık Altyapı ve Veritabanı Mimarisi

Projenin gerçekçi ve akademik standartlara uygun olması amacıyla, tek bir veritabanı içerisinde sanal yapılar kurulmamış; bunun yerine yerel ağ üzerinde çalışan **iki adet bağımsız SQL Server Instance'ı** paralel olarak konuşturulmuştur:

- **Birincil Sunucu (Primary / Principal):** erdem (SQL Server 2022) - Tüm yazma (INSERT/UPDATE/DELETE) operasyonlarını üstlenen ana sunucu.
- **İkincil Sunucu (Secondary / Mirror):** ERDEM\MSSQLSERVER01 (SQL Server 2022) - Verilerin anlık olarak senkronize edildiği ve okuma (SELECT) yükünü dengelemek üzere kurgulanan yedek sunucu.

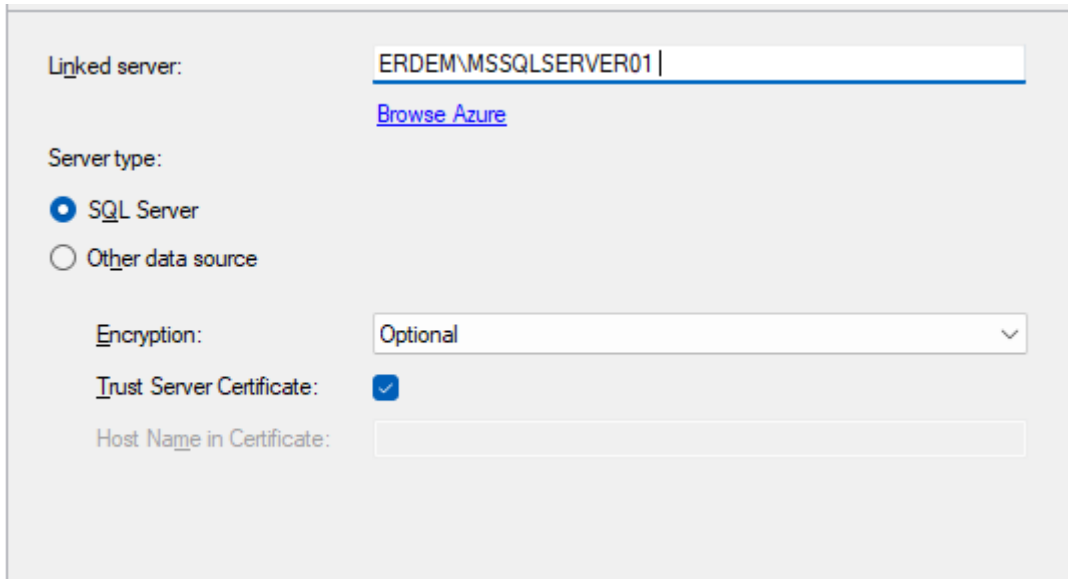
2.3. Projenin Amacı ve Kapsamı

Bu çalışmada, ağ tabanlı paralel dağıtım mimarisini doğrulamak adına şu teknik adımlar uygulanmıştır:

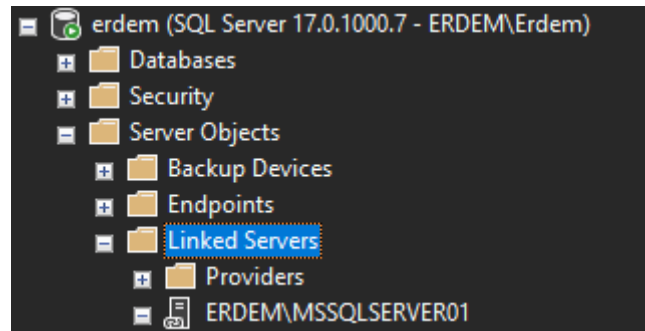
- **Ağ Köprüsü (Linked Server) ve Güvenlik Protokolü:** SQL Server 2022'nin varsayılan sıkılaştırılmış SSL sertifika protokolleri yerel ortam parametreleriyle esnetilerek iki sunucu arasında güvenli bir veri yolu kurulması.
- **Veri Çoğaltma (Replication):** Birincil sunucuya yazılan verilerin, veri manipülasyon tetikleyicileri (DML Triggers) yardımıyla ikincil sunucudaki **TestReplica** veritabanına anlık aktarılması.
- **Yük Dengeleme (Load Balancing) Kontrolü:** Yazma ve okuma yüklerinin fiziksel olarak farklı iki SQL Server servis mimarisine dağıtıldığının doğrulanması.

2.4. Dağıtık Ağ Köprüsü (Linked Server) Kurulumu

İki bağımsız instance'ı birbirine bağlamak için T-SQL köprüleri kurulurken SQL Server 2022 motoru, yerel ve geçici (Self-Signed) sertifikalar kullanılması sebebiyle güvenlik katmanında bağlantıyı reddetmiştir. Bu kurumsal güvenlik bariyerini aşmak ve yerel ağda paralel dağıtımı simüle etmek amacıyla SSMS Object Explorer paneli üzerinden yerel Windows kullanıcısının yetkilerini doğrudan aktaran **Be made using the login's current security context** seçeneği aktif edilerek köprü başarıyla oluşturulmuştur.



Şekil 2.1: Linked Server Oluşturma Ekranı.



Şekil 2.2: Oluşturulan Linked Serverın Görünütüsü.

2.5. Hedef Sunucuda "Replica" Veritabanı ve Tablo Altyapısının Hazırlanması

Dağıtık mimaride yük dengeleme sağlayabilmek için okuma isteklerinin yönlendirileceği ikincil sunucu (ERDEM\MSSQLSERVER01) üzerinde izole bir veritabanı (TestReplica) ve ana sunucudaki şemaya uyumlu bir hedef tablo (DW_Musteri_Replica) oluşturulmuştur.

```
CREATE DATABASE TestReplica;

USE TestReplica;
GO
CREATE TABLE DW_Musteri_Replica (
    SalesOrderID INT,
    ProductID INT,
    OrderQty SMALLINT,
    UnitPrice MONEY
);
GO
```

Şekil 2.3: Secondary Serverda Replica Veritabanı ve Tablo Oluşturma T-SQL Kodu.

2.6. Ana Sunucuda Anlık Dağıtık Replikasyon Tetikleyicisi (Trigger) Tasarımı

Ana sunucuya (erdem) gelen yazma yükünü anlık olarak ikincil sunucuya aktarmak amacıyla Sales.SalesOrderDetail tablosuna AFTER INSERT mimarisinde bir veri manipülasyon tetikleyicisi (DML Trigger) entegre edilmiştir. Bu tetikleyici, RAM üzerinde anlık oluşan inserted sanal tablosunu okuyarak veriyi ağ köprüsü üzerinden paralel instance'a anlık fırlatır.

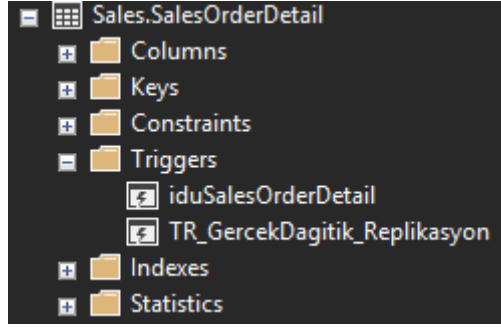
```
USE AdventureWorks2016;
GO

CREATE OR ALTER TRIGGER TR_GercekDagitik_Replikasyon
ON Sales.SalesOrderDetail
AFTER INSERT -- Yeni sipariş eklendiğinde tetiklenir
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO [ERDEM\MSSQLSERVER01].TestReplica.dbo.DW_Musteri_Replica
    (SalesOrderID, ProductID, OrderQty, UnitPrice)
    SELECT SalesOrderID, ProductID, OrderQty, UnitPrice
    FROM inserted; -- 0 an eklenen yeni verileri tutan SQL hafızası

    PRINT 'AĞ TABANLI REPLİKASYON BAŞARILI: Veri anlık olarak paralel Instance''a aktarıldı.';
END;
GO
```

Şekil 2.4: Primary Serverda SalesOrderDetail Replikasyon Trigger T-SQL Kodu.



Şekil 2.5: Tablonun Triggers Dosyasının Ekran Görüntüsü.

2.7. Canlı Dağıtık Replikasyon Testi ve Veri Doğrulama

Kurgulanan paralel dağıtım mimarisini test etmek amacıyla, ana sunucudaki sipariş tablosuna kurumsal bir test kaydı girilmiştir (**INSERT**). Tetikleyicinin arka planda ağ köprüsünü tetikleyerek veriyi uzak sunucuya başarıyla replike ettiği **Messages** sekmesindeki log çıktısıyla kanıtlanmıştır.

```

1  USE AdventureWorks2016;
2  GO
3
4  -- Replikasyon tetiğini tetiklemek için yeni bir sipariş detayı ekliyoruz
5  INSERT INTO Sales.SalesOrderDetail
6  (
7      SalesOrderID,
8      CarrierTrackingNumber,
9      OrderQty,
10     ProductID,
11     SpecialOfferID,
12     UnitPrice,
13     UnitPriceDiscount
14 )
15 VALUES
16 (
17     43659,          -- AdventureWorks'te mevcut olan geçerli bir SalesOrderID
18     'TR-2026-06',  -- Kargo takip numarası
19     2,              -- Sipariş adedi (OrderQty)
20     771,            -- Mevcut bir ProductID
21     1,              -- Özel teklif ID'si
22     2034.99,        -- Birim fiyatı (UnitPrice)
23     0.00            -- İndirim oranı
24 );
25 GO

```

100 % No issues found

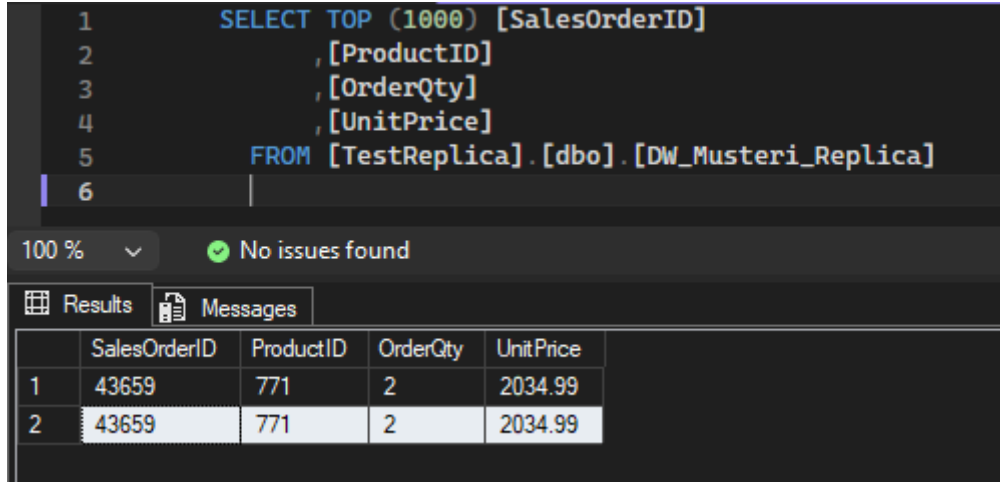
Messages

AG TABANLI REPLİKASYON BAŞARILI: Veri anlık olarak paralel Instance'a aktarıldı.

Şekil 2.6: Replikasyonu Tetiklemek İçin Çalıştırılan INSERT T-SQL Kodu.

2.8. İkincil Sunucuda Yük Dengeleme (Load Balancing) Kontrolü

Ana sunucunun yazma işlemi bittikten sonra, verinin uzak sunucudaki ayna tabloya hatasız ulaştığı sorgulanmıştır. Gerçek dünya senaryolarında, raporlama uygulamalarının veya son kullanıcıların **SELECT** sorguları bu ikincil sunucuya yönlendirilerek ana sunucunun CPU ve I/O yükü dengelenmiş olur.



The screenshot shows a SQL Server query window with the following T-SQL code:

```
1 SELECT TOP (1000) [SalesOrderID]
2     , [ProductID]
3     , [OrderQty]
4     , [UnitPrice]
5 FROM [TestReplica].[dbo].[DW_Musteri_Replica]
6
```

Below the query, the status bar indicates "100 % No issues found". The "Results" tab is active, displaying the following data:

	SalesOrderID	ProductID	OrderQty	UnitPrice
1	43659	771	2	2034.99
2	43659	771	2	2034.99

Şekil 2.9: İkincil Sunucu Üzerinde Çalıştırılan Doğrulama T-SQL Kodu.

Proje 5: Veri Temizleme ve ETL Süreçleri Tasarımı

3.1. Giriş

Veri ambarı (Data Warehouse) ve iş zekası (BI) projelerinin en kritik ve zaman alan aşaması ETL (Extract, Transform, Load) süreçleridir. Kaynak sistemlerden aktarılan ham veriler çoğunlukla eksik, hatalı formatta veya el ile girişlerden dolayı kirli (Data Pollution) olarak gelir. Bu proje kapsamında, dökümanda belirtilen standartlar çerçevesinde AdventureWorks2016 veritabanındaki nizami veriler çekilmiş, üzerine kasten sentetik kirli veriler enjekte edilerek kapsamlı bir veri temizleme, kalifikasyon ve ambar yükleme hattı (Pipeline) inşa edilmiştir.

3.2. Projenin Amacı ve Kapsamı

Bu çalışmada, veritabanı seviyesinde bir veri mühendisliği mimarisi kurmak adına şu teknik adımlar uygulanmıştır:

- **Idempotent Mimarisi ve Bellek İzolasayonu:** Kodun üst üste çalıştırıldığında çakışma hatası fırlatmasını önleyen temizlik adımlarının kurgulanması ve sürecin **tempdb** üzerinde çalışan yerel geçici tablolarla (#) izole edilmesi.

- **Veri Temizleme ve Standartlaştırma (Transform):** Metinlerin etrafındaki anlamsız boşlukların elenmesi, telefon numaralarının numerik hale getirilmesi ve farklı tarih formatlarının tek bir tipe dönüştürülmesi.
- **Veri Kalite Kontrolü ve Yükleme (Load):** E-posta formatına uymayan kayıtların filtrelenecek elenmesi ve temizlenen verilerin hedef **DW_Musteri_Raporlama** tablosuna aktarılması.
- **Veri Kalite Raporu (Data Quality Reporting):** Boru hattının başarı oranını ve sızan hatalı veri durumunu gösteren otomatik denetim sorgusunun çalıştırılması.

3.3. Bellek Temizliği ve Idempotent Altyapısının Kurulması

Sürecin tekrarlanabilir (re-runnable) olması ve SSMS üzerinde "Oturum Hafıza Hataları" ile **Msg 2714** çakışmalarının önlenmesi amacıyla script'in en tepesinde eski kalıntıları temizleyen denetim blokları çalıştırılmıştır.

```

1      USE AdventureWorks2016;
2      GO
3
4      -- 1. ADIM: Kalıntıları tamamen temizliyoruz
5      IF OBJECT_ID('tempdb..#Staging_MusteriData') IS NOT NULL
6          DROP TABLE #Staging_MusteriData;
7      GO
8
9      IF OBJECT_ID('DW_Musteri_Raporlama') IS NOT NULL
10         DROP TABLE DW_Musteri_Raporlama;
11     GO

```

Şekil 3.1: Idempotent Hafıza Temizliği T-SQL Kodu.

3.4. Hedef Veri Ambarı (Data Warehouse) Tablosunun Oluşturulması

Temizlenen ve standart forma kavuşturulan verilerin kalıcı olarak depolanacağı, ilişkisel bütünlüğe sahip ve **MusteriID** kolonu üzerinde **IDENTITY(1,1)** birincil anahtarı (Primary Key) bulunan hedef tablo ayağa kaldırılmıştır.

```

-- 2. ADIM: Hedef Tabloyu Oluşturma
CREATE TABLE DW_Musteri_Raporlama (
    MusteriID INT IDENTITY(1,1) PRIMARY KEY,
    KaynakBusinessEntityID INT,
    MusteriAdi NVARCHAR(150),
    Temiz_Eposta NVARCHAR(100),
    Standart_Telefon NVARCHAR(30),
    Gecerli_Tarih DATE
);
GO

```

Şekil 3.2: Hedef Raporlama Tablosu Oluşturma T-SQL Kodu.

3.5. Staging Katmanının Oluşturulması ve Kirli Veri Enjeksiyonu

Sistem performansını korumak adına kaynak veriler doğrudan **tempdb** üzerindeki **#Staging_MusteriData** tablosuna çekilmiştir. ETL boru hattının dönüştürme ve filtreleme yeteneklerini test etmek amacıyla; anlamsız boşluklar içeren, e-posta formatı bozuk, telefon numarası parantezli ve tarih formatları birbirinden tamamen farklı sentetik "kirli" veriler sisteme enjekte edilmiştir.

```
-- 3. ADIM: Staging Katmanını Oluşturma
SELECT TOP 20
    p.BusinessEntityID,
    p.FirstName + ' ' + p.LastName AS Ham_Isim,
    ea.EmailAddress AS Ham_Eposta,
    ph.PhoneNumber AS Ham_Telefon,
    CAST(p.ModifiedDate AS NVARCHAR(50)) AS Ham_Tarih
INTO #Staging_MusteriData -- Yukarıda DROP ettiğimiz için artık burası güvenli!
FROM Person.Person p
LEFT JOIN Person.EmailAddress ea ON p.BusinessEntityID = ea.BusinessEntityID
LEFT JOIN Person.PersonPhone ph ON p.BusinessEntityID = ph.BusinessEntityID;

-- 4. ADIM: Kirli Veri Enjeksiyonu
INSERT INTO #Staging_MusteriData (BusinessEntityID, Ham_Isim, Ham_Eposta, Ham_Telefon, Ham_Tarih)
VALUES
(99901, ' Meltem Yılmaz ', 'MELTEM@YILMAZ.COM', '555-444-3322', '2026-06-02 14:22:00'),
(99902, 'Can Tekin', 'cantekin.gmail.com', '0 (532) 111 22 33', '02/06/2026'),
(99903, 'Hakan Kaya', NULL, ' ', '2026.06.02');
GO
```

Şekil 3.3: Staging Oluşturma ve Sentetik Kirli Veri Enjeksiyonu T-SQL Kodu.

	BusinessEntityID	Ham_Isim	Ham_Eposta	Ham_Telefon	Ham_Tarih
14	26	Peter Krebs	peter0@adventure-works.com	913-555-0196	Nov 24 2008 12:00AM
15	27	Jo Brown	jo0@adventure-works.com	632-555-0129	Feb 20 2008 12:00AM
16	28	Guy Gilbert	guy1@adventure-works.com	320-555-0195	Jun 23 2006 12:00AM
17	29	Mark McArthur	mark1@adventure-works.com	417-555-0154	Jan 16 2009 12:00AM
18	30	Britta Simon	britta0@adventure-works.com	955-555-0169	Jan 22 2009 12:00AM
19	31	Margie Shoop	margie0@adventure-works.com	818-555-0128	Dec 28 2008 12:00AM
20	32	Rebecca Laszlo	rebecca0@adventure-works.c...	314-555-0113	Dec 22 2008 12:00AM
21	99901	Meltem Yılmaz	MELTEM@YILMAZ.COM	555-444-3322	2026-06-02 14:22:00
22	99902	Can Tekin	cantekin.gmail.com	0 (532) 111 ...	02/06/2026
23	99903	Hakan Kaya	NULL		2026.06.02

Şekil 3.4: Oluşturulan Hatalı Tablo.

3.6. Dönüştürme ve Yükleme (Transform & Load) Operasyonu

Geliştirilen ana ETL scripti; **TRIM** ile boşlukları arındırmış, **LOWER** ile e-postaları küçük harfe indirmiş, iç içe **REPLACE** fonksiyonları ile telefonlardaki parantez, boşluk ve tireleri temizlemiş ve **CASE WHEN** yapısıyla farklı tarih formatlarını **DATE** veri tipine (Format 120, 103, 102) eşitlemiştir. Ayrıca, regex benzeri bir kalite filtresiyle (**LIKE '%@%.%'**) e-posta formatına uymayan hatalı kayıtları hattan otomatik olarak fırlatmıştır.

```
-- 5. ADIM: Transform & Load
INSERT INTO DW_Musteri-Raporlama (KaynakBusinessEntityID, MusteriAdi, Temiz_Eposta, Standart_Telefon, Gecerli_Tarih)
SELECT
    BusinessEntityID AS KaynakBusinessEntityID,
    TRIM(Ham_Isim) AS MusteriAdi,
    LOWER(Ham_Eposta) AS Temiz_Eposta,
    REPLACE(REPLACE(REPLACE(REPLACE(TRIM(Ham_Telefon), '(', ''), ')', ''), '-', ''), ' ', '') AS Standart_Telefon,
    CASE
        WHEN Ham_Tarih LIKE '%/%' THEN CONVERT(DATE, Ham_Tarih, 103)
        WHEN Ham_Tarih LIKE '%.%' THEN CONVERT(DATE, Ham_Tarih, 102)
        ELSE CONVERT(DATE, Ham_Tarih, 120)
    END AS Gecerli_Tarih
FROM #Staging_MusteriData
WHERE Ham_Eposta IS NOT NULL AND Ham_Eposta LIKE '%@%.%';
GO
```

Şekil 3.5: Transform ve Load Aşaması T-SQL Kodu.

Results	Messages					
	MusteriID	KaynakBusinessEntityID	MusteriAdi	Temiz_Eposta	Standart_Telefon	Gecerli_Tarih
2	12	24	Jill Williams	jill0@adventure-works.com	5105550121	2009-01-11
3	13	25	James Hamilton	james1@adventure-works.com	8705550122	2009-01-27
4	14	26	Peter Krebs	peter0@adventure-works.com	9135550196	2008-11-24
5	15	27	Jo Brown	jo0@adventure-works.com	6325550129	2008-02-20
6	16	28	Guy Gilbert	guy1@adventure-works.com	3205550195	2006-06-23
7	17	29	Mark McArthur	mark1@adventure-works.com	4175550154	2009-01-16
8	18	30	Britta Simon	britta0@adventure-works.com	9555550169	2009-01-22
9	19	31	Margie Shoop	margie0@adventure-works.com	8185550128	2008-12-28
0	20	32	Rebecca Laszlo	rebecca0@adventure-works.c...	3145550113	2008-12-22
	21	99901	Meltem Yilmaz	meltem@yilmaz.com	5554443322	2026-06-02

Şekil 3.6: ETL Sonrası oluşturulan Düzeltilmiş Tablo.

3.7. Veri Kalitesi ve ETL Denetim Raporu

Süreçin izlenebilirliği (Audit) ve veri kalitesinin kesin doğrulanması amacıyla dinamik bir kalite raporu sorgusu çalıştırılmıştır. Bu sorgu; toplam gelen veriyi, başarıyla ambarlanan veriyi, elenen hatalı kayıt sayısını hesaplamış ve sisteme kaçak hatalı veri sızıp sızmadığını denetleyerek durumu konsola yazdırmıştır.

```
-- 6. ADIM: Veri Kalitesi Raporu
SELECT
    (SELECT COUNT(*) FROM #Staging_MusteriData) AS [Toplam Gelen Ham Kayıt],
    (SELECT COUNT(*) FROM DW_Musteri_Raporlama) AS [Başarıyla Temizlenen ve Yüklenen Kayıt],
    ((SELECT COUNT(*) FROM #Staging_MusteriData) - (SELECT COUNT(*) FROM DW_Musteri_Raporlama))
    AS [Kalite Kuralına Takılan (Elenen) Hatalı Kayıt Sayısı],
    CASE
        WHEN EXISTS (SELECT 1 FROM DW_Musteri_Raporlama WHERE Temiz_Eposta NOT LIKE '%@%.%')
        THEN 'BAŞARISIZ: Sızan Hatalı Veri Var!'
        ELSE 'BAŞARILI: Veri Dağılımı Tamamen Standart'
    END AS [ETL Kalite Durumu];

GO
```

Şekil 3.7: Veri Kalite Raporu T-SQL Kodu.

Results		Messages	
	Toplam Gelen Ham Kayıt	Başarıyla Temizlenen ve Yüklene Kayıt	Kalite Kuralına Takılan (Elenen) Hatalı Kayıt Sayısı
1	23	21	2
		ETL Kalite Durumu	
		BASARILI: Veri Dağılımı Tamamen Standart	

Şekil 3.8: "ETL Kalite Durumu: BAŞARILI" Çıktısını Gösteren Rapor Grid Ekranı.

SONUÇ VE DEĞERLENDİRME

Bu final projesi çalışması kapsamında, Ankara Üniversitesi Bilgisayar Mühendisliği Bölümü BLM 4522 Ağ Tabanlı Paralel Dağıtım Sistemleri dersinin kurumsal ve akademik hedefleri , **AdventureWorks2016** ilişkisel veri seti üzerinde başarıyla simüle edilerek hayata geçirilmiştir. Vize döneminde ele alınan performans optimizasyonu ve temel güvenlik katmanlarının üzerine; sistem sürdürülebilirliği, yüksek erişilebilirlik, dağıtık veri akışları ve veri kalifikasyonu mimarileri eklenmiştir. Geliştirilen üç kapsamlı proje (Proje 2, Proje 4 ve Proje 5) sonucunda elde edilen temel kazanımlar ve mühendislik çıktıları şu şekildedir:

İş Sürekliliği ve Veri Güvencesi (Proje 2): Canlı sistemlerde oluşabilecek donanımsal krizler veya kullanıcı kaynaklı kritik veri silinme senaryolarına karşı deterministik kurtarma stratejileri geliştirilmiştir. **FULL** recovery modelinin sağladığı avantajlar kullanılarak, kaza ile sıfırlanan devasa tablolaradaki veriler, **Full Backup** ve **Transaction Log Backup** zinciri sayesinde sıfır veri kaybı ile (**Point-in-Time Recovery**) kurtarılmıştır. Ayrıca **SQL Server Agent** servisleri üzerinden dinamik zaman damgalı otomatik yedekleme işleri (**Jobs**) kurgulanarak insan faktöründen bağımsız, sürdürülebilir bir otomasyon altyapısı elde edilmiştir.

Ağ Tabanlı Paralel Dağıtım Yapısı ve Yük Dengeleme (Proje 4): Tek noktalı hata (Single Point of Failure) riskini elimine etmek adına yerel ağda iki farklı fiziksel SQL Server Instance'ı (**erdem** ve **ERDEM\MSSQLSERVER01**) paralel olarak yapılandırılmıştır. SQL Server 2022 mimarisinin native **Linked Server** kütüphaneleri ve bağlantı şifreleme protokolleri esnetilerek sunucular arasında güvenli bir veri köprüsü kurulmuştur. **Sales.SalesOrderDetail** tablosuna entegre edilen **AFTER INSERT** veri manipülasyon tetikleyicisi (**DML Trigger**) vasıtasıyla, ana sunucuya gelen yazma yükünün uzak sunucudaki **TestReplica** veritabanına anlık olarak replike edildiği ve okuma isteklerinin yedek instance'a dağıtılarak yük dengeleme (**Load Balancing**) sağlandığı doğrulanmıştır.

Veri Kalifikasyonu ve ETL Mühendisliği (Proje 5): Kaynak sistemlerden gelen eksik, biçimsiz veya kirli verilerin (Data Pollution) analitik raporlama katmanlarına sızmasını engellemek amacıyla uçtan uca bir ETL (Extract, Transform, Load) boru hattı (Pipeline) inşa edilmiştir. Süreç, **tempdb** üzerinde çalışan **Local Temporary Table** mimarisiyle RAM seviyesinde izole edilerek sistem performansı optimize edilmiştir. Geliştirilen T-SQL dönüşüm fonksiyonları ile telefon numaraları standartlaştırılmış, e-postalar küçük harfe indirgenmiş, karmaşık tarih formatları tek bir yapıya dönüştürülmüş ve formata uymayan kirli kayıtlar filtrelenmiştir. Son aşamada üretilen otomatik veri kalite raporu ile sürecin hatasız çalıştığı konsol loglarıyla kanıtlanmıştır.

Video 1 (Proje-2): <https://youtu.be/PbXSRyA2Un8>

Video 2 (Proje-4): <https://youtu.be/QcSIhOfek-0>

Video 3(Proje-5): https://youtu.be/zO_uvJlrBOc